

Reviewer Pack — Minimal p-Adic Order of Boolean Functions and AC0 Circuit Lo...

Entry 27383ddef2d4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 23:27:30 UTC

Minimal p-Adic Order of Boolean Functions and AC0 Circuit Lower Bounds

Entry ID: 27383ddef2d4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 23:27:30 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): AC⁰ Circuit Complexity

Statement:

[‘For all boolean functions f , the minimal p-adic order of their Fourier coefficients, denoted as $\omega(f)$, is lower bounded by the depth of any AC⁰ circuit computing f , i.e., $\omega(f) = \Theta(2^d)$ where d is the minimum depth of an AC⁰ circuit for f .’, ‘This bound holds uniformly across all boolean functions for a fixed p-adic field.’, ‘If there exists an AC⁰ function g that does not satisfy $\omega(g) = \Theta(2^d)$, then g cannot be computed by any constant-depth AC⁰ circuit.’]

Rationale (proposer’s reasoning):

[‘p-adic analysis provides a framework to study the complexity of boolean functions in terms of their Fourier coefficients, which have been studied less frequently than other analytical tools like Fourier analysis over the real or complex numbers.’, ‘The proposed invariant $\omega(f)$ is expected to reveal deep connections between p-adic arithmetic and circuit complexity by providing lower bounds for AC⁰ circuits.’, ‘If this conjecture is true, it would offer a novel approach to proving circuit lower bounds and provide insights into the structure of AC⁰ functions.’]

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fcacc3024582b686

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean p-adic order $\omega(f)$ of boolean functions f matches the depth d of their AC⁰ circuits within a threshold of 3 standard deviations, across at least 24 out of 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic analysis AND AC0 circuit complexity - Fourier coefficients in p-adic field AND AC0 depth lower bounds - boolean functions p-adic order AND AC0 circuit depth relationship

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def fourier_coefficients(f, p):
        n = len(f)
        coeffs = [0] * (n + 1)
        for k in range(n + 1):
            sum_val = 0
            for x in range(2**n):
                sum_val += f[x] * math.exp(-2j * math.pi * k * x / p) / (2**n)
            coeffs[k] = sum_val
        return coeffs

    def ac0_circuit_depth(f, n):
        # Simplified AC0 circuit depth calculation for demonstration purposes
        # This is a placeholder and should be replaced with actual AC0 circuit construction logic
        return random.randint(1, 6)

    p = 2**32 # Example p-adic field
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
```

```

    f = generate_boolean_function(n)
    coeffs = fourier_coefficients(f, p)
    omega_f = max(abs(coeff) for coeff in coeffs[1:])
    depth = ac0_circuit_depth(f, n)

    total_metric_value += omega_f
    instances_tested += 1

mean_metric_value = total_metric_value / instances_tested
std_metric_value = math.sqrt(sum((omega_f - mean_metric_value)**2 for omega_f in coeffs[1:]) / instances_tested)

conjecture_holds = abs(mean_metric_value - 2**depth) <= 3 * std_metric_value

return {
    "metric_name": "p-adic order",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"omega(f) = {mean_metric_value}, depth = {depth}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2**i + 1 for i in range(5, 8)] # First 30 prime numbers

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_support_fraction support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6e501fc4.py", line 73, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6e501fc4.py", line 46, in run_trial
    coeffs = fourier_coefficients(f, p)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6e501fc4.py", line 30, in fourier_coeff
    sum_val += f[x] * math.exp(-2j * math.pi * k * x / p) / (2**n)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

TypeError: must be real number, not complex

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture’s support conditions. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16605
2	preregistration	ollama_remote	glm4:latest	0	0	9563
3	novelty	ollama_remote	glm4:latest	0	0	8904
4	novelty	ollama_remote	glm4:latest	0	0	8801
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42344
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10396
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9894
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9537
9	judge	ollama_remote	glm4:latest	0	0	23073

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 139117 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/27383ddef2d4.jsonl)

12. Reproducibility

- To reproduce this cycle:
- 1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
 - 2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
 - 3. The full LLM transcripts are in research/audit/27383ddef2d4.jsonl for verification of provenance
 - 4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/27383ddef2d4.tar.gz (if generated)